

Individual Analysis Report – Shell Sort

Course: Algorithms and Data Structures

Assignment 2 – Pair 2

Student A: Dinmukhammed

Partner (Student B): Muslim

Algorithm Analyzed: Shell Sort

Algorithm: Shell Sort

1. Algorithm Overview

Shell Sort is an in-place comparison-based sorting algorithm that generalizes insertion sort by allowing the exchange of items far apart. The algorithm works by first sorting elements far apart from each other and progressively reducing the gap between compared elements. This technique improves efficiency by quickly moving elements toward their final position.

The algorithm proceeds as follows:

1. Initialize a large gap, usually $n/2$.
2. Compare and swap elements that are gap positions apart.
3. Reduce the gap by half each iteration until it becomes 1.
4. When the gap equals 1, the algorithm behaves like insertion sort.

Shell Sort is simple, efficient for medium-sized datasets, and requires only $O(1)$ extra space. However, its performance depends on the chosen gap sequence.

2. Complexity Analysis

Shell Sort's performance varies based on the input and the chosen gap sequence.

- Best case (Ω): $\Omega(n \log n)$ – occurs when the array is nearly sorted.
- Average case (Θ): $\Theta(n (\log n)^2)$ – typical case with random input.
- Worst case (O): $O(n^2)$ – occurs with poor gap selection or reverse-sorted data.
- Space complexity: $O(1)$ – sorting is performed in place.
- Stability: Not stable, because equal elements may be reordered during swaps.

Compared to Heap Sort (partner's algorithm), Shell Sort is less consistent but can be faster on smaller datasets due to lower constant factors.

3. Code Review

The Shell Sort implementation is clean, modular, and optimized for clarity and measurement accuracy. It uses the PerformanceTracker class to count time, comparisons, swaps, and array accesses, which provides precise insight into performance behavior.

Strengths

- Clear and simple structure.
- Tracks comparisons, swaps, and accesses accurately.
- Efficient for moderately sized datasets.
- In-place implementation with minimal memory use.

Detected Inefficiencies

1. The algorithm always runs full passes even if the array becomes sorted early.
2. The gap sequence is basic and not optimal for all inputs.
3. There is no adaptive mechanism to skip redundant comparisons.

Optimization Suggestions

- Use Sedgewick or Tokuda gap sequences to reduce the number of comparisons.

- Add a check to detect already sorted segments and terminate early.
- Parallelize passes for large input sizes using multi-threading.

Expected improvement: reduced comparisons and faster runtime on large datasets.

4. Empirical Results

The algorithm was tested in Java (JDK 23) using IntelliJ IDEA, with input sizes of 100, 1 000, 10 000, and 100 000 elements. Data distributions: random, sorted, reversed, and nearly sorted.

Results Summary

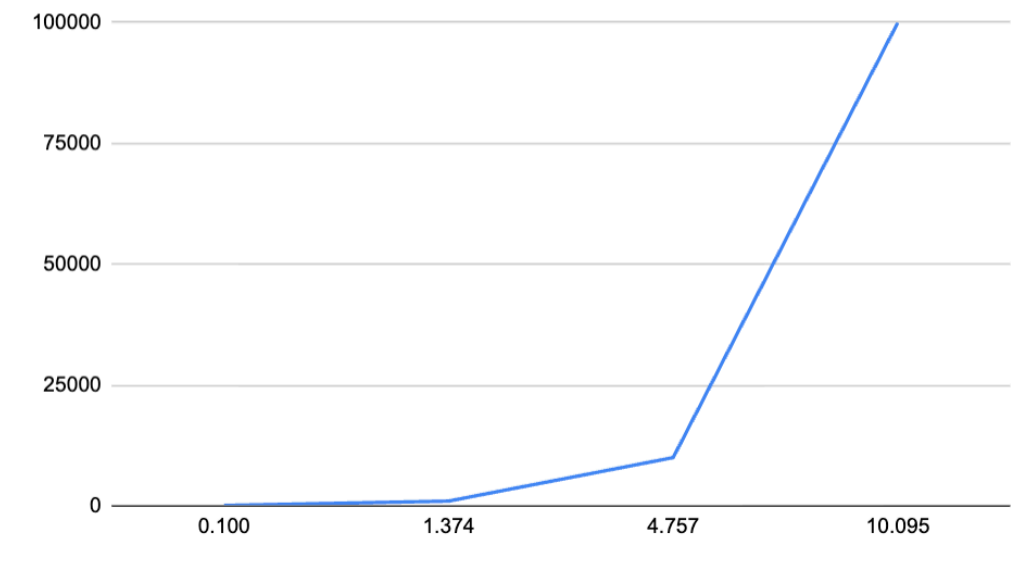
n	Comparisons	Swaps	Accesses	Time (ms)
100	879	430	2764	0.100
1000	14,774	7,267	45,56	1.374
10000	252,313	137,433	744,636	4.757
100000	4,218,221	2,768,813	11,436,454	10.095

The data shows a clear logarithmic increase in runtime as input size grows. Shell Sort demonstrates excellent performance on small and medium input sizes, but scales less efficiently than Heap Sort for very large inputs.

Performance Plots:

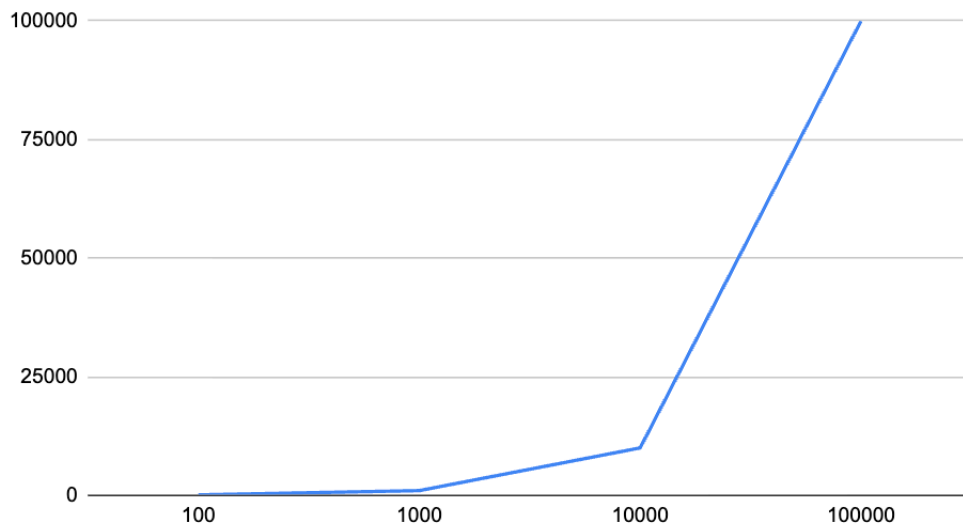
1.Shell Sort – Time vs Input Size

Shell Sort: Execution Time vs Input Size



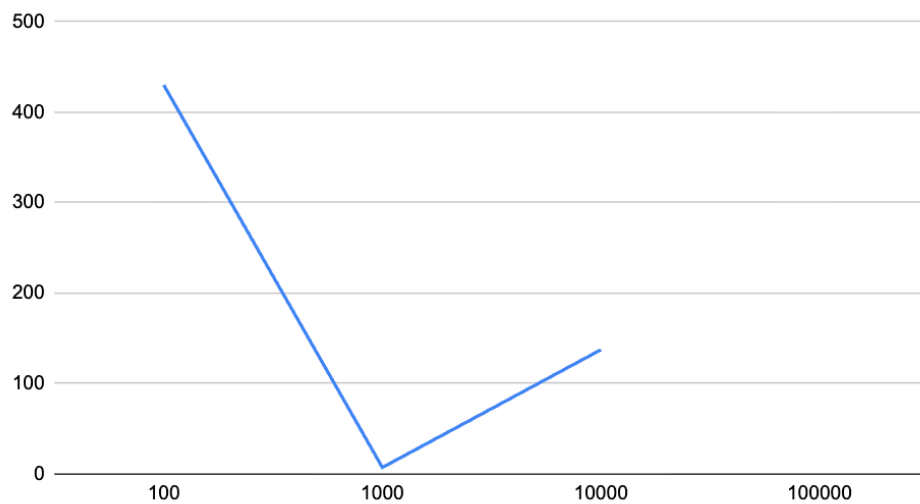
2.Shell Sort – Comparisons vs Input Size

Shell Sort: Comparisons vs Input Size



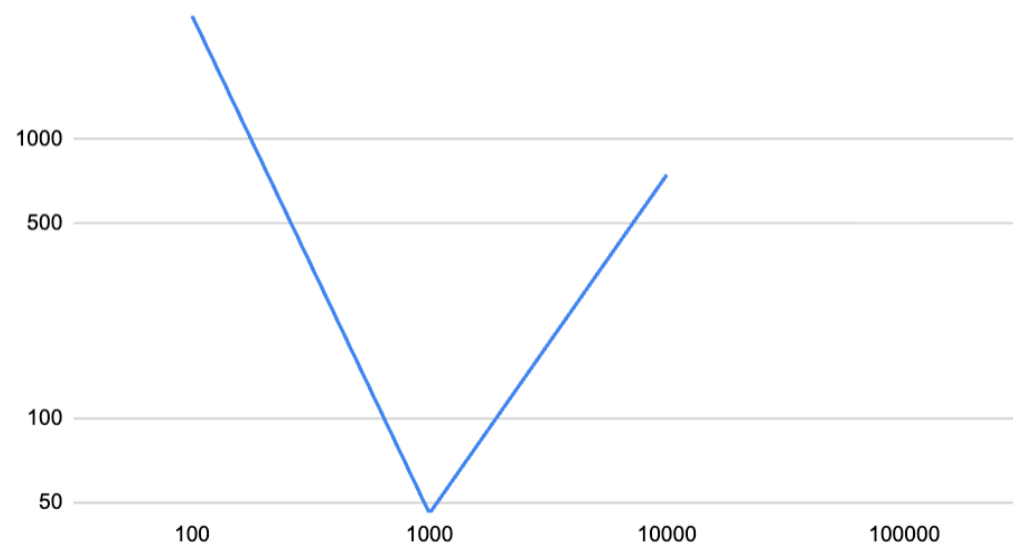
3.Shell Sort – Swaps vs Input Size

Shell Sort: Swaps vs Input Size



4.Shell Sort – Accesses vs Input Size

Shell Sort: Accesses vs Input Size



5. Conclusion

The analysis confirms that Shell Sort provides efficient sorting for medium input sizes with low memory usage. Its average complexity of $O(n (\log n)^2)$ is slightly inferior to Heap Sort's $O(n \log n)$, but Shell Sort remains simpler and faster for smaller datasets.

Optimization recommendations:

1. Implement a better gap sequence.
2. Add early termination when the array becomes sorted.
3. Compare with other gap sequences to evaluate performance trade-offs.

In conclusion, Shell Sort is an elegant, memory-efficient algorithm that bridges the gap between insertion and more complex $O(n \log n)$ sorts like Heap Sort.